Configuration과 Synthesis

halo2에서 Circuit의 핵심은 `configure()` 와 `synthesize()` 메서드입니다. configure는 회로의 구조와 제약조건을 정의하고, synthesize는 실제 witness 값들을 회로에 할당합니다. 이 두 단계의 분리는 halo2의 핵심 설계 철학 중 하나입니다.

Configuration 단계

configure() 함수의 역할

```
fn configure(meta: &mut ConstraintSystem<F>) -> Self::Config;
```

configure 함수는 회로의 "청사진"을 만드는 단계입니다. 이 단계에서는:

1. **컬럼 정의**: advice, instance, fixed 컬럼들을 생성
2. **게이트 정의**: 제약조건을 표현하는 커스텀 게이트 생성
3. **설정 반환**: 생성된 구성 요소들을 Config 타입으로 반환

컬럼 생성

```
fn configure(meta: &mut ConstraintSystem<F>) -> Self::Config {
// 다양한 타입의 컬럼 생성
let advice = meta.advice_column(); // witness 값 저장용
let instance = meta.instance_column(); // public input 저장용
let fixed = meta.fixed_column(); // 상수 값 저장용
```

```
let selector = meta.complex_selector(); // 게이트 활성화용

// 등식 제약조건 활성화 (필요한 경우)

meta.enable_equality(advice);

meta.enable_equality(instance);

// ... 게이트 정의

}
```

게이트 정의

게이트는 회로의 제약조건을 정의합니다. 다음은 간단한 덧셈 게이트 예제입니다:

```
// "step" 게이트: next = curr + 1

meta.create_gate("step", |meta| {

let curr = meta.query_advice(advice, Rotation::cur());

let next = meta.query_advice(advice, Rotation::next());

let q_enable = meta.query_selector(q_enable);

// 제약조건: q_enable * (curr - next + 1) = 0

// 즉, q_enable이 1일 때 next = curr + 1

vec![q_enable * (curr - next + Expression::Constant(F::ONE))]

});
```

Config 타입 설계

Config 타입은 configure에서 생성된 모든 구성 요소를 저장합니다:

```rust
#[derive(Clone, Debug)]
struct TestConfig<F: Field + Clone> {
_ph: PhantomData<F>,
q_enable: Selector, // 게이트 활성화용 셀렉터
advice: Column<Advice>, // witness 값 저장용 컬럼
}
```

중요 사항:

- `Clone + Debug` trait 구현 필수
- 모든 컬럼과 셀렉터를 포함해야 함
- synthesize에서 사용할 모든 구성 요소 저장

Synthesis 단계

synthesize() 함수의 역할

```rust
fn synthesize(
&self,
config: Self::Config,
mut layouter: impl Layouter<F>,
) -> Result<(), Error>;
```

synthesize 함수는 실제 witness 값들을 회로에 할당하는 단계입니다:

1. **Region 생성**: 제약조건이 적용될 영역 정의

2. **값 할당**: witness 값들을 적절한 셀에 할당

3. **게이트 활성화**: 정의된 제약조건을 특정 행에서 활성화

Region 기반 할당

```
fn synthesize(
&self,
config: Self::Config,
mut layouter: impl Layouter<F>,
) -> Result<(), Error> {
layouter.assign_region(
|| "steps", // region 이름 (디버깅용)
|mut region| {
// 이 region 내에서 값 할당과 게이트 활성화
// ...
Ok(())
},
)?;
Ok(())
}
```

값 할당과 게이트 활성화

```
layouter.assign_region(
```

```
|| "steps",
|mut region| {
for i in 0..STEPS {
// 1. witness 값을 advice 컬럼에 할당
region.assign_advice(
|| "assign advice",
config.advice,
i, // 행 번호
|| self.values.as_ref().map(|values| values[i]),
)?;


// 2. 해당 행에서 게이트 활성화
config.q_enable.enable(&mut region, i)?;
}
// 마지막 값 할당 (next 값)
region.assign_advice(
|| "assign advice",
config.advice,
STEPS,
|| self.values.as_ref().map(|values| values[STEPS]),
)?;
Ok(())
},
```

```
)
```

실제 구현 예제: 덧셈 회로

다음은 연속된 값들이 1씩 증가하는 것을 검증하는 회로입니다:

```
use std::marker::PhantomData;
use halo2_proofs::{
circuit::{Layouter, SimpleFloorPlanner, Value},
plonk::{Advice, Circuit, Column, ConstraintSystem, Error, Expression, Selector},
poly::Rotation,
};
use ff::Field;

const STEPS: usize = 5;

// 회로 구조체 (witness 데이터 포함)
struct TestCircuit<F: Field> {
_ph: PhantomData<F>,
values: Value<Vec<F>>, // witness 값들
}

// 설정 구조체
```

```rust
#[derive(Clone, Debug)]
struct TestConfig<F: Field + Clone> {
_ph: PhantomData<F>,
q_enable: Selector,
advice: Column<Advice>,
}

impl<F: Field> Circuit<F> for TestCircuit<F> {
type Config = TestConfig<F>;
type FloorPlanner = SimpleFloorPlanner;

fn without_witnesses(&self) -> Self {
TestCircuit {
_ph: PhantomData,
values: Value::unknown(), // witness 제거
}
}

// Configuration 단계
fn configure(meta: &mut ConstraintSystem<F>) -> Self::Config {
let q_enable = meta.complex_selector();
let advice = meta.advice_column();
```

```
// 게이트 정의: next = curr + 1
meta.create_gate("step", |meta| {
let  curr = meta.query_advice(advice, Rotation::cur());
let  next = meta.query_advice(advice, Rotation::next());
let  q_enable = meta.query_selector(q_enable);
vec![q_enable * (curr - next + Expression::Constant(F::ONE))]
});


TestConfig {
_ph: PhantomData,
q_enable,
advice,
}
}


// Synthesis 단계
fn  synthesize(
&self,
config: Self::Config,
mut  layouter: impl Layouter<F>,
) -> Result<(), Error> {
layouter.assign_region(
|| "steps",
```

```
|mut region| {
// STEPS번 반복하여 값 할당과 게이트 활성화
for i in 0..STEPS {
region.assign_advice(
|| "assign advice",
config.advice,
i,
|| self.values.as_ref().map(|values| values[i]),
)?;
config.q_enable.enable(&mut region, i)?;
}

// 마지막 값 할당
region.assign_advice(
|| "assign advice",
config.advice,
STEPS,
|| self.values.as_ref().map(|values| values[STEPS]),
)?;

Ok(())
},
)?;
```

```
Ok(())
}
}
```

사용 예제

```
fn main() {
use halo2_proofs::halo2curves::bn256::Fr;
use halo2_proofs::dev::MockProver;

// witness 데이터 생성
let start = Fr::from(1337u64);
let mut values = vec![start];
while values.len() < STEPS + 1 {
let last = values.last().unwrap();
values.push(last + Fr::ONE); // 1씩 증가
}

// 회로 인스턴스 생성
let circuit = TestCircuit::<Fr> {
_ph: PhantomData,
values: Value::known(values),
};
```

```
// 검증
let  prover = MockProver::run(8, &circuit, vec![]).unwrap();
prover.verify().unwrap();
println!("회로 검증 성공!");
}
```

설계 패턴과 모범 사례

1. 단계별 분리

- **Configuration**: 회로 구조만 정의, witness 값 사용 금지
- **Synthesis**: 실제 값 할당, 구조 변경 금지

2. 오류 처리

```
// 값 할당 시 적절한 오류 메시지 제공
region.assign_advice(
|| "meaningful description", // 디버깅에 도움되는 설명
config.advice,
row,
|| value.ok_or(Error::Synthesis), // 적절한 오류 처리
)?;
```

3. Value 타입 활용

```
// witness 값은 Value<T>로 래핑
struct Circuit<F> {
secret: Value<F>, // private witness
public: F, // public input (Value 불필요)
}

// without_witnesses에서 적절히 처리
fn  without_witnesses(&self) -> Self {
Circuit {
secret: Value::unknown(), // witness 제거
public: self.public, // public 값은 유지
}
}
```

4. 모듈화된 설계

```
// 복잡한 회로는 여러 Config로 분리
#[derive(Clone, Debug)]
struct ComplexConfig {
adder_config: AdderConfig,
multiplier_config: MultiplierConfig,
```

```
hash_config: HashConfig,

}
```

다음 단계

Configuration과 Synthesis의 기본 개념을 이해했다면:

- Chip과 Region: 모듈화된 회로 설계 방법
- Layouter와 FloorPlanner: 효율적인 레이아웃 관리

이 두 단계의 명확한 분리는 halo2의 핵심 설계 원칙입니다. Configuration에서는 회로의 "무엇을"을 정의하고, Synthesis에서는 "어떻게"를 구현합니다.